

De Imagen a Grafo: OpenCV + NumPy + BFS

Esta guía muestra el flujo de trabajo completo: cómo tomar una imagen (ej. un mapa o laberinto), procesarla con OpenCV para convertirla en una matriz binaria, aplicar BFS para encontrar el camino más corto y dibujar la solución.

1. Preprocesamiento (OpenCV y NumPy)

El primer paso es simplificar la imagen. Las imágenes a color tienen 3 canales (RGB), lo cual es innecesario para lógica de rutas. Las pasamos a escala de grises y luego las "binarizamos" (blanco y negro puro).

```
import cv2
import numpy as np

# 1. Leer la imagen directamente en escala de grises
img = cv2.imread('mapa.png', cv2.IMREAD_GRAYSCALE)

# 2. Binarizar la imagen (Thresholding)
# Todo píxel más oscuro que 128 se vuelve 0 (Negro), el resto 255 (Blanco)
_, umbral = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

# 3. Convertir a Matriz lógica para nuestro algoritmo (usando NumPy)
# Asumimos que las líneas negras son paredes (1) y el blanco es camino libre (0)
grid = np.where(umbral == 0, 1, 0)
```

2. Búsqueda del Camino Más Corto (BFS)

Con la matriz lista, aplicamos nuestro BFS. A diferencia de solo contar la distancia, en este código guardamos la "historia" del camino para poder dibujarlo al final.

```
from collections import deque

def resolver_bfs(grid, inicio, meta):
    # grid.shape en NumPy nos da (filas, columnas)
    filas, columnas = grid.shape

    # La cola guarda tuplas de: (coordenada_actual, camino_recorrido_hasta_aqui)
```

```

cola = deque([(inicio, [inicio])])

visitados = set()
visitados.add(inicio)

# Arriba, Abajo, Izquierda, Derecha
direcciones = [(-1, 0), (1, 0), (0, -1), (0, 1)]

while cola:
    (f, c), camino = cola.popleft()

    # Si llegamos a la meta, retornamos la lista de coordenadas
    if (f, c) == meta:
        return camino

    for df, dc in direcciones:
        nf, nc = f + df, c + dc

        # 1. Dentro de los límites
        # 2. Es un camino libre (grid == 0)
        # 3. No lo hemos visitado
        if 0 <= nf < filas and 0 <= nc < columnas:
            if grid[nf, nc] == 0 and (nf, nc) not in visitados:
                visitados.add((nf, nc))
                cola.append((nf, nc), camino + [(nf, nc)]))

return None # No hay ruta posible

```

3. Visualización de la Solución (OpenCV)

Una vez que el algoritmo nos devuelve la lista de coordenadas que forman el camino más corto, usamos OpenCV para modificar la imagen original a color y trazar la ruta.

```

# Definimos puntos de inicio (fila, columna)
inicio = (10, 10)
meta = (150, 150)

# Llamamos a nuestro BFS
camino_optimo = resolver_bfs(grid, inicio, meta)

if camino_optimo:
    print("¡Camino encontrado!")
    # Volvemos a cargar la imagen, esta vez a color (para pintar en rojo)
    img_color = cv2.imread('mapa.png', cv2.IMREAD_COLOR)

```

```
for (f, c) in camino_optimo:
    # OpenCV usa formato (B, G, R) para los colores.
    # Modificamos el pixel a Rojo Puro.
    img_color[f, c] = [0, 0, 255]

# Guardamos la nueva imagen con la solución trazada
cv2.imwrite('solucion_mapa.png', img_color)
print("Imagen guardada como solucion_mapa.png")
else:
    print("No se encontró ningún camino válido.")
```

⚠ Concepto Clave para OpenCV y NumPy:

En matemáticas y en estas bibliotecas, las matrices se acceden mediante `matriz[filas, columnas]` (y, x). Esto es diferente al plano cartesiano normal (x, y). Siempre piensa primero en el eje vertical (Y) y luego en el horizontal (X) al recorrer las imágenes.